Beadandó I.

Feladat leírása:

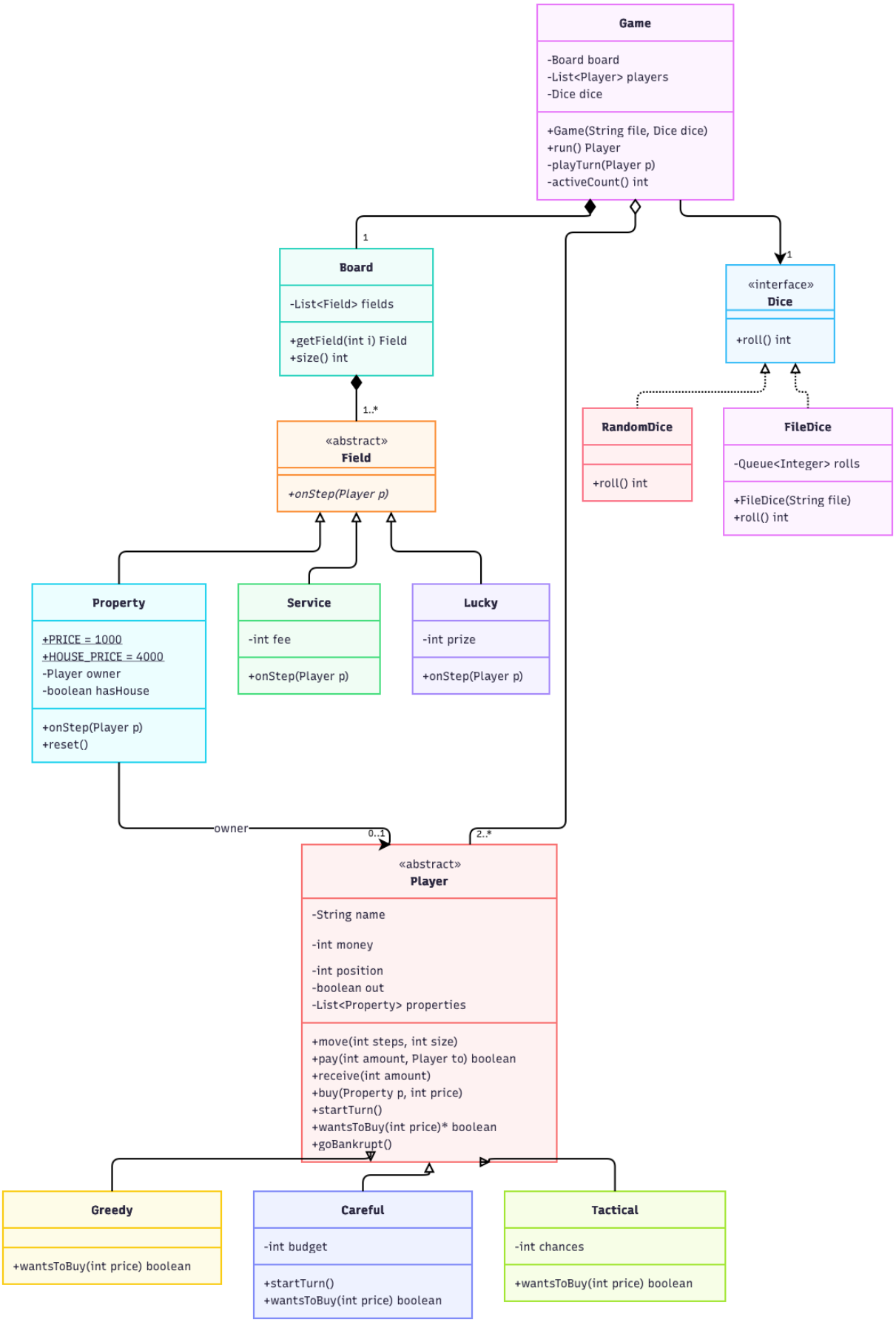
Szimuláljuk az alábbi egyszerűsített Capitaly társasjátékot! Adott néhány eltérő stratégiájú játékos és egy körpálya, amelyen különféle mezők sorakoznak egymás után. A pályát körbe-körbe újra és újra bejárják a játékosok úgy, hogy egy kockával dobva mindig annyit lépnek, amennyit a kocka mutat. A mezők három félék lehetnek: ingatlanok, szolgáltatások és szerencse mezők. Az ingatlant meg lehet vásárolni 1000 Petákért, majd újra rálépve házat is lehet rá építeni 4000 Petákért. Ha ezután más játékos erre a mezőre lép, akkor a mező tulajdonosának fizet: ha még nincs rajta ház, akkor 500 Petákot, ha van rajta ház, akkor 2000 Petákot. A szolgáltatás mezőre lépve a banknak kell befizetni a mező paramétereként megadott összeget. A szerencse mezőre lépve a mező paramétereként megadott összegű pénzt kap a játékos. Háromféle stratégiájú játékos vesz részt a játékban.

Kezdetben mindenki kap egy induló tőkét (10000 Peták), majd a „mohó” játékos ha egy még gazdátlan ingatlan mezőjére lépett, vagy övé az ingatlan, de még nincs rajta ház, továbbá van elég tőkéje, akkor vásárol. Az „óvatos” játékos egy körben csak a tőkéjének a felét vásárolja el, a „taktikus” játékos minden második vásárlási lehetőséget kihagyja. Ha egy játékosnak fizetnie kell, de nincs elégendő pénze, akkor kiesik a játékból, házai elvesznek, ingatlanjai megvásárolhatókká válnak.

A játék paramétereit egy szövegfájlból olvassuk be. Ez megadja a pálya hosszát, majd a pálya egyes mezőit. Minden mezőről megadjuk annak típusát, illetve ha szolgáltatás vagy szerencse mező, akkor annak pénzdíját. Ezt követően a fájl megmutatja a játékosok számát, majd sorban minden játékos nevét és stratégiáját. A tesztelhetőséghez fel kell készíteni a megoldó programot olyan szövegfájl feldolgozására is, amely előre rögzített módon tartalmazza a kockadobások eredményét.

Adjuk meg, melyik játékos nyeri meg a játékot és mekkora vagyona (mennyi a tőkéje, milyen ingatlanokat birtokol) van ekkor!

Osztálydiagram:



Metódusok leírása:

Game

- Game(String file, Dice dice): Konstruktor, amely beolvassa a paraméterfájlt (a pálya hosszát, a mezőket, a játékosok számát, nevét és stratégiáját), felépíti a Board-ot és a Player objektumokat, és eltárolja a kapott kockát. Hibás fájl esetén kivételt dob.
- run() Player: A játék fő ciklusa. Sorban mindenkit léptet a playTurn() segítségével, és addig fut, amíg egyetlen aktív játékos marad (vagy elfogynak a fájlból olvasott dobások). A győztest adja vissza.
- playTurn(Player p): Egy játékos egy köre. Ha a játékos már kiesett, kihagyja. Különben meghívja a startTurn()-t, dob a kockával, lépteti a játékost a move() metódussal, majd a Board-tól lekért mezőn meghívja az onStep(p)-t.
- activeCount() int: Visszaadja, hány játékos van még játékban. A run() ezzel ellenőrzi a játék végét.

Board

- getField(int i) Field: Visszaadja az i-edik pozíción lévő mezőt.
- size() int: A pálya hossza. A körbejáráshoz (maradékos osztáshoz) kell.

Field (absztrakt)

- onStep(Player p): Absztrakt metódus, amely leírja, mi történik, ha a játékos erre a mezőre lép. Minden leszármazott saját módon valósítja meg.

Property

- onStep(Player p): Ha a mező gazdátlan, megkérdezi a játékost (wantsToBuy(PRICE)), és igen esetén eladja neki. Ha a játékosé, de nincs rajta ház, ugyanígy felajánlja a házépítést HOUSE_PRICE-ért. Ha másé, a játékos a pay() segítségével fizet a tulajdonosnak: ház nélkül 500, házzal 2000 Petákot.
- reset(): Törli a tulajdonost és a házat. Akkor hívódik meg, amikor a tulajdonos kiesik, így az ingatlan újra megvásárolható lesz.

Service

- onStep(Player p): A játékos a pay(fee, null) hívással befizeti a banknak a mező díját. Ha nincs elég pénze, kiesik.

Lucky

- onStep(Player p): A játékos a receive(prize) hívással megkapja a mező nyereményét.

Player (absztrakt)

- move(int steps, int size): Lépteti a játékost: position = (position + steps) % size.

- pay(int amount, Player to) boolean: Ha van elég pénz, levonja az összeget, és ha a to nem null, jóváírja a kedvezményezettnek (null esetén a bank kapja). Ha nincs elég pénz, meghívja a goBankrupt()-ot, és false-t ad vissza.
- receive(int amount): Növeli a játékos tőkéjét.
- buy(Property p, int price): Levonja az árat, és ingatlanvásárlásnál felveszi az ingatlant a properties listába. Házépítésnél a ház megépül.
- startTurn(): Kör eleji előkészítés. Az alaposztályban üres, a Careful felüldefiniálja.
- wantsToBuy(int price) boolean: Absztrakt metódus, a stratégia lényege: eldönti, vásárol-e a játékos az adott áron.
- goBankrupt(): Kiesteti a játékost (out = true), minden ingatlanján meghívja a reset()-et, és kiüríti a listáját.

Greedy

- wantsToBuy(int price): Igaz, ha money >= price, vagyis mindig vásárol, ha teheti.

Careful

- startTurn(): A kör elején beállítja a keretet: budget = money / 2.
- wantsToBuy(int price): Csak akkor vásárol, ha price <= budget. Vásárlás után a keretet csökkenti az árral.

Tactical

- wantsToBuy(int price): Minden vásárlási lehetőségnél növeli a chances számlálót, és csak minden második alkalommal vásárol (például ha a számláló páros), feltéve, hogy van elég pénze.

Dice, RandomDice, FileDice

- roll() int (interfész): Egy kockadobás eredményét adja vissza (1–6).
- RandomDice.roll(): Véletlen számot generál 1 és 6 között.
- FileDice(String file): Beolvassa a fájlból az előre rögzített dobásokat egy sorba (Queue).
- FileDice.roll(): Kiveszi és visszaadja a sor következő elemét. Ha a sor kiürült, kivételt dob, vagy jelzi a játéknak, hogy vége a szimulációnak.

Tesztek:

1. Teljes játék a kézzel levezetett példán (GameTest.keziPeldaEredmenye)

Cél: Ellenőrzi, hogy a teljes szimuláció (beolvasás, dobás, lépés, vásárlás, házépítés, díjfizetés, szerencse mező, győztes kiválasztása) helyes eredményt ad.

Bemenet: A jatek.txt pálya (Property, Service 800, Property, Lucky 1500, Property, Service 2000), három játékos (Peti – mohó, Anna – óvatos, Bela – taktikus). A dobások fájlból jönnek: 2 4 2 2 2 2 4 3 2.

Elvárt eredmény: A dobások elfogyása után Anna nyer 11 000 Petákkal és 2 ingatlannal. Petinek 5000 Peták marad, és házat épített az ingatlanára. Belának 8500 Peták marad, ingatlanja nincs.

2. Bérleti díj házas ingatlanon (PropertyTest.berletiDijHazzal)

Cél: Ellenőrzi az ingatlan mező teljes életútját: vásárlás, házépítés, majd díjfizetés egy másik játékosnak.

Bemenet: Két mohó játékos, Peti és Anna, egy gazdátlan ingatlan. Peti kétszer rálép, utána Anna lép rá.

Elvárt eredmény: Peti 1000-ért megveszi az ingatlant, majd 4000-ért házat épít rá. Anna 2000 Peták díjat fizet neki. Végül Anna tőkéje 8000, Petié 7000 Peták.

3. Az óvatos játékos költési kerete (StrategyTest.ovatosKeretetCsokkentiAVasarlas)

Cél: Ellenőrzi, hogy az óvatos játékos egy körben legfeljebb a tőkéje felét költi el, és a vásárlások csökkentik ezt a keretet.

Bemenet: Egy óvatos játékos 10 000 Petákkal. A kör elején a kerete 5000. Ebben a körben 4000-ért vásárol.

Elvárt eredmény: A maradék keret 1000. Egy 1000 Petákos vásárlást még vállal, egy 1500 Petákosat már nem, pedig a tőkéje elég lenne hozzá.

4. Kiesés és az utolsó aktív játékos győzelme (GameTest.utolsoAktivJatekosNyer)

Cél: Ellenőrzi, hogy aki nem tud fizetni, kiesik. Az ingatlanjai ilyenkor újra gazdátlanok lesznek, és a játék véget ér, ha egy játékos marad.

Bemenet: Kétmezős pálya (Property, Service 20000), két mohó játékos (A és B). Dobások: 2 2 1.

Elvárt eredmény: A megveszi az ingatlant. B rálép, és 500 Peták díjat fizet A-nak. Ezután A a szolgáltatás mezőre lép, a 20 000 Petákos díjat nem tudja kifizetni, ezért kiesik, és az ingatlanja felszabadul. A győztes B, 9500 Petákkal.

5. Hibás bemeneti fájl kezelése (GameTest.ismeretlenMezotipus és GameTest.ervenytelenDobas)

Cél: Ellenőrzi, hogy a program felismeri a hibás bemenetet, és nem fut tovább hibás adatokkal.

Bemenet: Az egyik esetben a paraméterfájlban ismeretlen mezőtípus szerepel („Hotel”). A másikban a dobásfájlban egy 1–6 tartományon kívüli érték van („3 7”).

Elvárt eredmény: Mindkét esetben IllegalArgumentException kivétel keletkezik. A főprogram ezt elkapja, és „Hibás bemenet: …” üzenetet ír ki.